A

Project Report

on

HANDIFY: A REAL-TIME SIGN LANGUAGE TO TEXT CONVERSION FOR EMPOWERING DIFFERENTLY-ABLED COMMUNICATION

submitted as partial fulfilment for the award of

BACHELOR OF TECHNOLOGY DEGREE

SESSION 2025-26

in

Computer Science and Engineering (Artificial Intelligence)

By

Saumya (2200291520159)
Vansh Sihania (2200291520185)
Vidit Sharma (2200291520187)
Sumit Chauhan (2200291520172)

Under the supervision of

Ms. Anjali Chauhan

KIET Group of Institutions, Ghaziabad

Affiliated to

Dr. A.P.J. Abdul Kalam Technical University, Lucknow

(Formerly UPTU)

May, 2026

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree of diploma of the university or other institute of higher learning, except where due to acknowledgement has been made in the text.

SAUMYA	2200291520159	Signature:
VANSH SIHANIA	2200291520185	Signature:
VIDIT SHARMA	2200291520187	Signature:
SUMIT CHAUHAN	2200291520172	Signature:

Date:

CERTIFICATE

This is to certify that the project report entitled **"HANDIFY: A Real-Time Sign Language to Text Conversion for Empowering Differently-Abled Communication"** which is submitted by SAUMYA, VANSH SIHANIA, VIDIT SHARMA and SUMIT CHAUHAN in partial fulfillment of the requirements for the award of the degree B. Tech. in Department of Computer Science & Engineering (Artificial Intelligence) of Dr. A.P.J. Abdul Kalam Technical University, Lucknow is a record of the candidates own work carried out by them under my supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Ms. Anjali Chauhan	**Dr. Rekha Kashyap**
(Assistant Professor, CSE(AI))	**(Dean, CSE(AI) and CSE(AI&ML))**
Date of Approval:	

ACKNOWLEDGEMENT

It gives us a great sense of pleasure to present the report of the B. Tech Project undertaken during B. Tech. Final Year. We owe special debt of gratitude to Ms. Anjali Chauhan, Assistant Professor, Department of Computer Science & Engineering (Artificial Intelligence), KIET Group of Institutions, Ghaziabad, for her constant support and guidance throughout the course of our work. Her sincerity, thoroughness and perseverance have been a constant source of inspiration for us. It is only her cognizant efforts that our endeavors have seen light of the day.

We also take the opportunity to acknowledge the contribution of Dr. Rekha Kashyap, Dean, Department of CSE(AI) and CSE(AI&ML), KIET Group of Institutions, Ghaziabad, for her full support and assistance during the development of the project. We also do not like to miss the opportunity to acknowledge the contribution of all the faculty members of the department for their kind assistance and cooperation during the development of our project. Last but not the least, we acknowledge our friends for their contribution in the completion of the project.

Date:

SAUMYA	2200291520159	Signature:
VANSH SIHANIA	2200291520185	Signature:
VIDIT SHARMA	2200291520187	Signature:
SUMIT CHAUHAN	2200291520172	Signature:

ABSTRACT

The increasing global population of individuals with hearing impairments has created a significant communication gap between the deaf community and the general population. Traditional methods such as human interpreters are often costly, limited in availability, and not suitable for everyday interactions. To address this challenge, this project presents "Handify", a computer vision-based system designed to convert American Sign Language (ASL) gestures into readable text in real time.

The proposed system follows a modular pipeline architecture that utilizes OpenCV for video capture and MediaPipe for efficient hand detection and extraction of 21 hand landmarks. These landmarks are transformed into normalized spatial features based on their relative positions, ensuring invariance to hand position and scale. The processed features are then used to train a Random Forest classifier for gesture recognition.

The system is trained on a custom dataset of approximately 10,000–11,000 labeled samples representing ASL alphabets and digits. Experimental results demonstrate that the system achieves an accuracy of approximately 91.7% under standard lighting conditions while maintaining real-time performance of around 15–20 frames per second (FPS) on standard consumer hardware.

Unlike sensor-based approaches or computationally intensive deep learning models, Handify offers a lightweight, cost-effective, and accessible solution without requiring specialized hardware. The system provides an efficient and user-friendly interface for real-time gesture recognition, thereby contributing to improved communication and inclusivity for differently-abled individuals.

TABLE OF CONTENTS

DECLARATION ……...ii
CERTIFICATE …….....iii
ACKNOWLEDGEMENTS ……..iv
ABSTRACT ……..v
LIST OF FIGURES ……..viii
LIST OF TABLES ……ix
LIST OF ABBREVIATIONS ……...x
CHAPTER 1: INTRODUCTION ……1
1.1 Background and Motivation ……....1
1.2 Problem Statement ……..1
1.3 Objectives of the project ……1
1.4 Scope and Significance ……..……. 2
1.5 Organization of the Report ……..……3
CHAPTER 2: LITERATURE REVIEW ……..4
2.1 Evolution of Sign Language Recognition ……4
2.2 Sensor-based vs. Vision-based Approaches ……....5
2.3 Prior Works and Comparative Study ……...6
2.4 Research Gap and Justification for Handify ……8
CHAPTER 3: PROPOSED METHODOLOGY ……..10
3.1 System Overview and Architecture ……10
3.2 Data Acquisition and Preprocessing ……...11
3.3 Hand Landmark Extraction (MediaPipe) ……12
3.4 Feature Engineering ……...13
3.5 Classification using Random Forest ……14
3.6 Workflow of the System ……16
3.7 Advantages of the Proposed System ……..18
CHAPTER 4: RESULTS AND DISCUSSION ……...20
4.1 Experimental Setup ……20
4.2 Performance Metrics and Accuracy ……...21
4.3 Real-Time Processing and Capability ……..22
4.4 Limitations and Challenges ……23
CHAPTER 5: CONCLUSIONS AND FUTURE SCOPE ……...24
5.1 Conclusion ……..24
5.2 Future Scope ……..26
REFERENCES ……28
APPENDICES …… ……29

RESEARCH PAPER ……....31
PROOF OF PUBLICATION ……...40
PLAGIARISM REPORT …….....41

LIST OF FIGURES

Figure No.	Description	Page No.
3.1	System Architecture Diagram	11
3.2	MediaPipe Hand Landmarks showing 21 key points of hand	12
3.3	Real-time Hand Landmark Detection using MediaPipe	13
3.4	Workflow of the System for Real-Time Gesture Recognition	16
B.1	Input hand gestures (top row) and their corresponding recognized outputs (bottom row).	30
B.2	Recognition of the word “What” using sequential hand gestures.	30

LIST OF TABLES

Table No.	Description	Page No.
2.1	Comparison of existing Sign Language Recognition Systems	7
3.1	Model Training Parameters	15
4.1	Experimental Setup and System Configuration	21
4.2	Summary of Classification Result	22

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ASL	American Sign Language
ML	Machine Learning
RF	Random Forest
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
HMM	Hidden Markov Model
FPS	Frames Per Second
HSV	Hue, Saturation, Value
UI	User Interface
ROI	Region of Interest
NLP	Natural Language Processing
HCI	Human-Computer Interaction
WHO	World Health Organization

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Communication is the cornerstone of human interaction, social integration, and cognitive development. However, according to the World Health Organization (WHO), over 466 million people worldwide suffer from disabling hearing loss. For this significant demographic, sign language serves as the primary mode of communication. While sign language is highly expressive and structured, it remains largely unintelligible to the general hearing population, creating a profound communication divide.

Traditionally, bridging this gap has relied on human interpreters. However, professional interpretation is costly, limits personal privacy, and is not scalable for everyday, ubiquitous interactions. This technological void motivated the development of automated Sign Language Recognition (SLR) systems. The motivation behind "Handify" is to democratize accessibility by utilizing widely available consumer hardware (such as laptop webcams) to provide an independent, real-time communication channel for differently-abled individuals.

1.2 Problem Statement

The core problem addressed in this project is the lack of an accessible, low-cost, and real-time mechanism to translate American Sign Language (ASL) into readable text. Existing solutions either require cumbersome sensory gloves that inhibit natural movement or rely on computationally heavy deep learning architectures that demand expensive Graphics Processing Units (GPUs). Therefore, there is a critical need for an efficient computer vision-based system that operates seamlessly on standard hardware without compromising accuracy.

1.3 Objective of the Project

The primary objectives of this project are as follows:

- To design and develop “Handify”, a vision-based real-time Sign Language Recognition system that converts hand gestures into readable text.

- To utilize OpenCV and MediaPipe for efficient real-time hand detection and extraction of 21 key hand landmarks.
- To generate normalized spatial features from the extracted landmark coordinates, ensuring invariance to hand position and scale.
- To implement and train a Random Forest classifier on a custom dataset consisting of approximately 10,000–11,000 labeled samples representing ASL alphabets and digits (0 - 9).
- To achieve accurate and efficient real-time gesture recognition on standard consumer hardware without requiring specialized devices or high computational resources.

1.4 Scope and Significance

The scope of the Handify system is currently focused on recognizing static American Sign Language (ASL) gestures, specifically alphabets from A to Z and digits from 0 to 9. The system operates as a one-way translation tool that converts sign language gestures into readable text in real time. It is designed to function on standard consumer hardware such as a laptop with a webcam, making it accessible and easy to deploy without the need for specialized devices or expensive computational resources.

Although the current implementation is limited to static gestures, the system is capable of forming simple words and sentences by sequentially combining recognized gestures. This makes it useful for basic communication scenarios where users can convey their messages effectively.

The significance of this project lies in its potential to contribute towards building an inclusive and accessible society. By reducing reliance on human interpreters, the system provides a level of independence to deaf and hard-of-hearing individuals. It enables them to communicate more freely in environments such as educational institutions, workplaces, and public service areas.

Furthermore, the system promotes the use of affordable and scalable technology solutions to address real-world problems. By eliminating the need for specialized hardware such as sensor-based gloves, Handify ensures that assistive technology can reach a wider population. Overall, the project highlights the role of technology in empowering differently-abled individuals and bridging communication gaps in society.

1.5 Organization of the Project Report

The report is organized in a structured and systematic manner to provide a clear and comprehensive understanding of the proposed system, its design, and implementation. Each chapter is arranged sequentially to guide the reader from the fundamental concepts to the final results and conclusions.

Chapter 1 presents the introduction of the project, including the background and motivation behind the development of the system. It also defines the problem statement, outlines the objectives, and discusses the scope and significance of the proposed solution.

Chapter 2 provides a detailed literature review, discussing existing sign language recognition systems and comparing different approaches such as sensor-based and vision-based methods. It also highlights the limitations of current systems and identifies the research gap that motivates the development of Handify.

Chapter 3 explains the proposed methodology in detail, including the overall system architecture and workflow. It covers important stages such as data acquisition, color space conversion, hand landmark extraction using MediaPipe, feature engineering through normalization of landmark coordinates, and classification using the Random Forest algorithm.

Chapter 4 presents the implementation and experimental results of the system. It includes details about the experimental setup, performance evaluation metrics, accuracy analysis under different conditions, and discussion of the system's real-time capabilities along with its limitations.

Chapter 5 concludes the project by summarizing the overall work and key contributions. It also outlines the future scope of the system, suggesting possible improvements such as dynamic gesture recognition, mobile deployment, and integration with advanced technologies for enhanced performance.

CHAPTER 2

LITERATURE REVIEW

2.1 Evolution of Sign Language Recognition

Sign Language Recognition (SLR) has gained significant importance in recent years as a key research area within Human-Computer Interaction (HCI). The primary objective of SLR systems is to reduce the communication gap between individuals who use sign language and those who do not understand it. As observed in existing studies, communication plays a crucial role in social interaction, education, and professional environments, and the inability to communicate effectively creates barriers for deaf and hard-of-hearing individuals.

In the early stages of SLR development, researchers focused on **sensor-based systems**, which required users to wear specialized hardware devices. These systems typically used instrumented gloves equipped with sensors such as flex sensors, accelerometers, and gyroscopes to capture hand movements and finger bending accurately. The advantage of such systems was their ability to provide precise and structured data, making gesture recognition relatively straightforward.

However, despite their accuracy, these systems had several drawbacks. They were expensive, uncomfortable to wear, and restricted natural hand movement. Additionally, they were not suitable for everyday use, as users had to rely on external hardware to communicate. This made them impractical for real-world scenarios where quick and seamless communication is required.

With advancements in technology, particularly in computer vision and digital imaging, the focus gradually shifted toward more natural and accessible solutions. Researchers began exploring **vision-based approaches**, which eliminated the need for wearable devices and allowed users to communicate using only a camera. This transition marked a significant milestone in the evolution of SLR systems, making them more user-friendly and scalable.

2.2 Sensor-based vs. Vision based Approaches

The comparison between sensor-based and vision-based approaches highlights a fundamental shift in the design philosophy of sign language recognition systems.

Sensor-based approaches rely on direct measurement of hand movements using hardware devices. These systems offer high accuracy because they capture exact finger positions and motion data. However, their dependence on external devices makes them less practical for widespread use. The requirement of wearing gloves or sensors not only increases cost but also reduces user comfort and flexibility.

On the other hand, **vision-based systems** use cameras to capture hand gestures and apply image processing techniques to interpret them. These systems are non-intrusive and allow users to communicate naturally without any physical constraints. As mentioned in the research paper, vision-based approaches have become more popular due to their affordability and ease of integration into devices such as smartphones and laptops.

Despite their advantages, vision-based systems face several challenges. Environmental factors such as lighting conditions, background complexity, and occlusion can affect the accuracy of gesture recognition. Variations in skin tone and hand orientation further complicate the detection process.

To overcome these challenges, modern systems use advanced preprocessing techniques. For example:

- **Background subtraction** is used to isolate moving objects from the background.
- **HSV color space segmentation** helps in detecting skin regions more effectively.
- **Morphological operations** such as erosion and dilation reduce noise and improve clarity.

These preprocessing steps ensure that the system can reliably detect hand regions before passing the data to machine learning models for classification.

2.3 Prior Works and Comparative Study

Over the years, several research studies have contributed to the development and improvement of sign language recognition systems. Each study has focused on improving accuracy, efficiency, or usability, but none have completely solved all challenges simultaneously.

One of the earliest and most influential works in this domain was conducted by **Starner et al.**, who developed a real-time ASL recognition system using **Hidden Markov Models (HMMs)**. Their system achieved an accuracy of approximately 92% for a limited set of gestures. However, it required controlled lighting conditions and the use of colored gloves, which limited its applicability in real-world environments.

With the advancement of machine learning, researchers began adopting **deep learning techniques** to improve performance. **Camgoz et al.** introduced neural network-based models capable of performing end-to-end sign language translation. These models significantly improved accuracy (around 94%) and enabled recognition of continuous sign sequences. However, they required large datasets and high computational resources, making them unsuitable for real-time applications on standard devices.

Further research by **Ong and Ranganath** emphasized the importance of combining spatial and temporal features for better recognition performance. Their work demonstrated that analyzing both hand shape and motion improves accuracy, although the system was limited in vocabulary size.

Building upon these ideas, **Koller et al.** developed hybrid models that combined CNNs and RNNs to capture both spatial and temporal information. These models achieved high accuracy levels of around 95% and were capable of handling large vocabularies. However, they required GPU-based training and long processing times, making them less accessible for users with basic hardware.

In addition to academic research, commercial systems such as **SignAll** have been developed to provide real-time sign language translation. These systems often use multiple cameras and advanced hardware setups to achieve high accuracy. However, they are expensive and not portable, which limits their adoption in everyday scenarios.

The comparative analysis presented in the research paper clearly shows that:

- High accuracy systems require expensive hardware and computational resources.
- Lightweight systems are more accessible but may compromise slightly on accuracy.

This comparison highlights the need for a balanced approach that can provide both efficiency and performance.

The above discussion highlights differences in accuracy, hardware requirements, and limitations across various approaches. A summarized comparison is presented below.

Authors	Approach/Algorithm	Hardware Requirement	Accuracy	Limitations
Starner et al.	Hidden Markov Model (HMM)	Colored Gloves, Camera	~92%	Requires controlled environment and wearable markers.
Camgoz et al.	Deep Learning (Neural Networks)	High-end GPU	~94%	High computational cost and large dataset requirement
Ong & Rangnath	Feature based Vision Techniques	Standard Camera	~88%	Limited vocabulary and less robustness
Koller et al.	CNN + RNN Hybrid Model	High-end GPU	~95%	Computationally expensive, requires long training time
SignAll System	Multi-Camera Vision System	Specialized Setup	~90%	Expensive and not portable
Proposed (Handify)	MediaPipe + Random Forest	Standard Webcam (No GPU)	~91.7%	Struggles in extremely low light; limited to static signs

Table 2.1 : Comparison of existing Sign Language Recognition Systems

2.4 Research Gap and Justification for Handify

The existing literature clearly highlights a significant gap between highly accurate sign language recognition systems and those that are practically deployable in real-world scenarios. While deep learning-based approaches have demonstrated remarkable accuracy and the ability to handle complex gesture patterns, they often require large-scale datasets, high computational power, and dedicated hardware such as GPUs. This makes them less accessible for everyday use, especially in low-resource environments. On the other hand, simpler machine learning approaches are more efficient and easier to deploy, but they may struggle to maintain robustness under varying environmental conditions such as changes in lighting, background complexity, and user variations.

This trade-off between accuracy and accessibility creates a strong need for a balanced solution that can deliver reliable performance while remaining computationally efficient and easy to deploy on standard consumer hardware. The Handify system is designed to address this exact challenge by combining efficient feature extraction techniques with a lightweight yet effective classification model.

The Handify system leverages MediaPipe for real-time hand tracking and landmark detection. MediaPipe is capable of detecting 21 key hand landmarks that represent important points such as the wrist, finger joints, and fingertips. These landmarks provide a structured and consistent representation of the hand, capturing essential spatial information such as finger positions and the overall configuration of the hand. This structured representation allows the system to focus on meaningful geometric patterns rather than relying on raw pixel data, thereby improving robustness and efficiency.

To further enhance consistency and reduce the impact of variations in hand position and scale, the extracted landmark coordinates are transformed into normalized spatial features. This normalization is achieved by adjusting the coordinates relative to the minimum values, ensuring that the feature representation remains invariant to translation and positioning within the frame. As a result, the system can generalize better across different users and environments without requiring complex preprocessing techniques.

For classification, the system employs a Random Forest algorithm, which is an ensemble learning technique known for its robustness and efficiency. Random Forest constructs multiple decision trees during training and combines their outputs using a majority voting mechanism. This approach not only improves prediction accuracy but also reduces the risk of overfitting. Additionally, Random Forest is well-suited for handling non-linear relationships in data and provides fast inference, making it highly appropriate for real-time applications.

The system is trained on a custom dataset consisting of approximately 10,000–11,000 labeled samples representing various American Sign Language (ASL) gestures, including alphabets and digits. Experimental evaluation shows that the Handify system achieves an accuracy of

approximately 91.7% while operating on standard hardware without requiring a dedicated GPU. This demonstrates that high performance can be achieved without relying on computationally expensive deep learning models.

Furthermore, the system performs reliably under different environmental conditions, including variations in lighting and background. Although performance may slightly decrease in extremely low-light scenarios, the overall system remains stable and functional for practical usage. By combining efficient landmark-based feature extraction with a lightweight classification model, Handify successfully bridges the gap between performance and accessibility.

In conclusion, the proposed system offers a practical, cost-effective, and scalable solution for real-time sign language recognition. It demonstrates that lightweight machine learning techniques, when combined with optimized frameworks such as MediaPipe, can effectively address real-world challenges and contribute significantly to the development of assistive technologies for differently-abled individuals.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 System Overview and Architecture

The Handify system is designed as a modular and efficient pipeline architecture that enables real-time sign language recognition using standard hardware. The primary objective of this architecture is to ensure smooth data flow, low latency, and reliable gesture recognition while maintaining computational efficiency. The system is organized into multiple interconnected modules, each responsible for a specific stage in the processing pipeline.

The first module, known as the Input Module, is responsible for capturing real-time video data from a standard webcam. This module continuously streams video frames, ensuring a consistent input for further processing. Each captured frame serves as the basic unit of data for the system.

The second module, the Detection Module, utilizes MediaPipe to detect and track the hand region directly from each frame. MediaPipe applies optimized machine learning models to identify hand landmarks without requiring explicit preprocessing techniques or manual Region of Interest (ROI) extraction. This simplifies the detection process while maintaining high accuracy.

The third module, the Recognition Module, performs feature extraction and classification. It converts the detected hand landmarks into normalized spatial features and feeds them into a machine learning model for gesture prediction. This module plays a crucial role in determining the overall performance and accuracy of the system.

Finally, the User Interface Module displays the recognized gesture output in real time. The predicted gesture is overlaid on the video stream, providing immediate visual feedback to the user and enabling seamless interaction.

The system follows a sequential pipeline architecture, where the output of one module serves as the input to the next. This design ensures smooth execution and enables real-time performance on standard consumer hardware without requiring specialized computational resources.

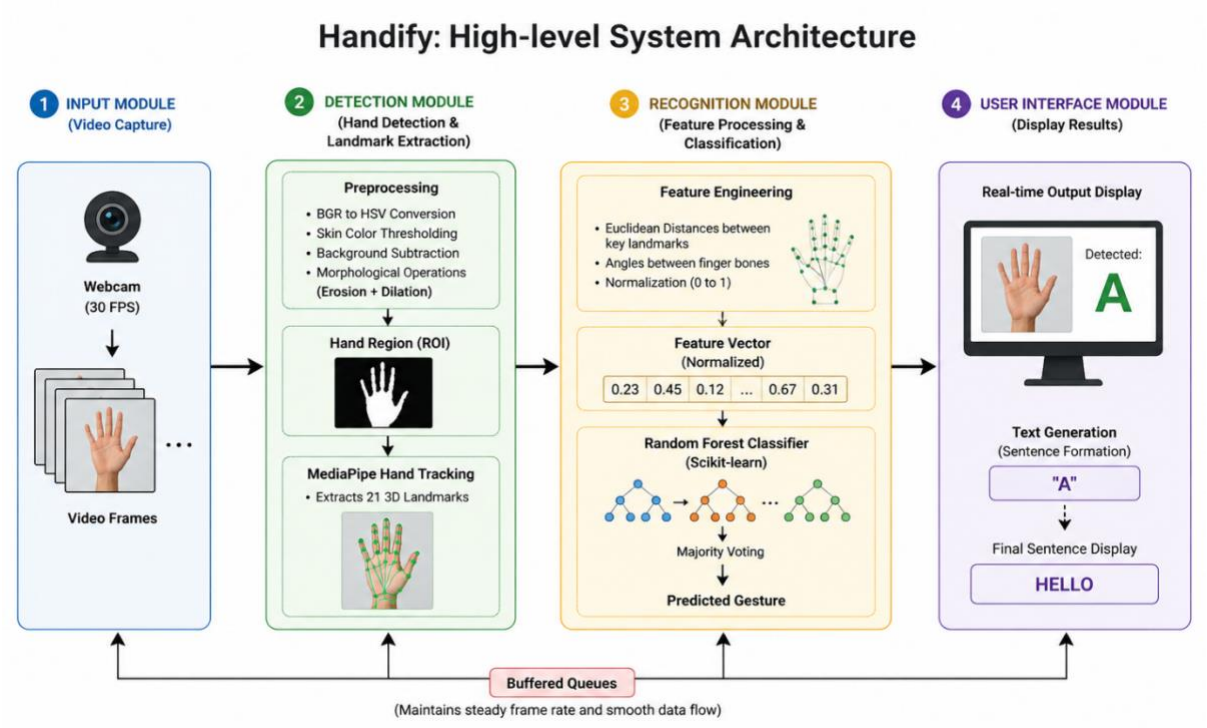


Figure 3.1: System Architecture Diagram

3.2 Data Acquisition and Preprocessing

The initial stage of the Handify system involves acquiring real-time input data using a standard webcam. The camera continuously captures video frames, providing a live stream of visual data for gesture recognition. Each frame is processed independently to ensure smooth execution and maintain real-time performance.

The captured frames are initially in the BGR color space, which is the default format used by OpenCV. However, for effective hand detection, MediaPipe requires the input frames to be in RGB format. Therefore, each frame is converted from BGR to RGB before being passed to the detection stage.

Unlike traditional approaches that rely on preprocessing techniques such as skin color segmentation, background subtraction, or morphological operations, the Handify system directly utilizes MediaPipe's built-in hand detection framework. MediaPipe employs optimized machine learning models to detect and track hand regions efficiently across varying lighting conditions and backgrounds.

This approach eliminates the need for explicitly defining contours or manually extracting the Region of Interest (ROI), thereby simplifying the overall processing pipeline. As a result, the system achieves improved computational efficiency while maintaining reliable and consistent hand detection. The processed frames are then forwarded to the next stage for hand landmark extraction, where hand landmarks are extracted for further feature computation and gesture classification.

3.3 Hand Landmark Extraction (MediaPipe)

Once the hand is detected, the system utilizes MediaPipe's hand tracking framework to extract detailed hand landmarks. MediaPipe employs a machine learning-based pipeline that detects and tracks hand features in real time with high precision.

The system identifies **21 distinct 3D landmarks** on the hand. These landmarks represent key anatomical points, including the wrist, finger joints, and fingertips. Each landmark is defined by its spatial coordinates (x, y, z), providing a structured representation of the hand's geometry.

One of the major advantages of using MediaPipe is its ability to maintain accuracy even under challenging conditions such as slight hand rotations, partial occlusions, and variations in hand orientation. The landmark-based representation ensures that the system captures the relative positioning of fingers rather than relying solely on pixel-level information.

Additionally, MediaPipe is optimized for real-time performance and can operate efficiently on devices without dedicated GPUs. This makes it highly suitable for the Handify system, which aims to provide accessibility without requiring high-end hardware.

The extracted landmarks form the foundation for the next stage of processing, where they are transformed into features suitable for classification.

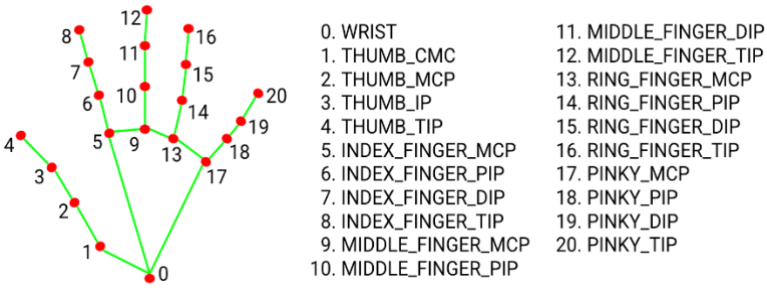


Figure 3.2: MediaPipe Hand Landmarks showing 21 key points of the hand

As shown in Figure 3.2, MediaPipe defines 21 key landmarks on the hand, including the wrist, finger joints, and fingertips, which together form a structured representation of hand geometry.

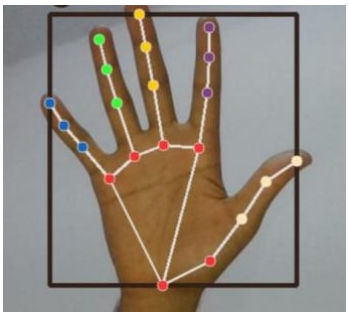


Figure 3.3: Real-time Hand Landmark Detection using MediaPipe

Figure 3.3 illustrates the real-time detection of these landmarks, where the key points are overlaid on the hand to track its movement dynamically. This combination of structured landmark representation and real-time visualization enables accurate and efficient gesture recognition.

3.4 Feature Engineering

The raw landmark coordinates obtained from MediaPipe cannot be directly used for classification due to variations in hand position, scale, and orientation. These variations can lead to inconsistencies in the data, making it difficult for the classifier to generalize effectively. Therefore, the Handify system applies a feature engineering process to transform raw spatial data into a normalized and consistent feature representation.

The feature extraction process involves computing normalized spatial features from the detected hand landmarks. For each landmark, the x and y coordinates are normalized by subtracting the minimum coordinate values among all landmarks. This ensures that the features are invariant to hand position and translation within the frame.

The normalized coordinates are then arranged into a structured feature vector, representing the relative configuration of the hand. This compact representation captures essential information about the hand's shape and finger positioning while eliminating unnecessary variations caused by absolute positioning.

This approach simplifies the feature extraction process while maintaining sufficient discriminative power for gesture recognition. The resulting feature vector is consistent, efficient, and well-suited for real-time classification tasks.

Finally, the normalized feature vector is passed to the classification module, where it serves as input to the Random Forest model for gesture prediction.

3.5 Classification using Random Forest

The classification stage of the Handify system is performed using the Random Forest algorithm, implemented through the Scikit-learn library. Random Forest is an ensemble learning technique that combines multiple decision trees to improve prediction accuracy and reduce the risk of overfitting.

During the training phase, the model is trained on a custom dataset consisting of approximately 10,000 – 11,000 labeled samples representing different hand gestures. Each sample corresponds to a feature vector derived from normalized hand landmark coordinates along with its associated gesture label.

The Random Forest model works by constructing multiple decision trees, where each tree is trained on a subset of the data. During prediction, each tree produces an output, and the final predicted gesture is determined through a majority voting mechanism. This approach enhances robustness and ensures stable performance even in the presence of minor variations in input data.

One of the key advantages of using Random Forest is its fast inference time, making it suitable for real-time applications. Additionally, it does not require high computational resources, allowing the system to operate efficiently on standard hardware.

During real-time execution, the system processes each frame independently and generates predictions continuously. The predicted gesture is updated dynamically based on the latest input, enabling smooth and responsive real-time gesture recognition.

The key parameters used during the training of the Random Forest model are summarized in Table 3.1.

PARAMETER	VALUE
Algorithm	Random Forest
Dataset Size	~10,000 – 11,000
Number of Classes	~38
Train – Test Split	80:20
Feature Type	Normalized Landmark Coordinates
Input Size	42 features

Table 3.1: Model Training Parameters

3.6 Workflow of the System

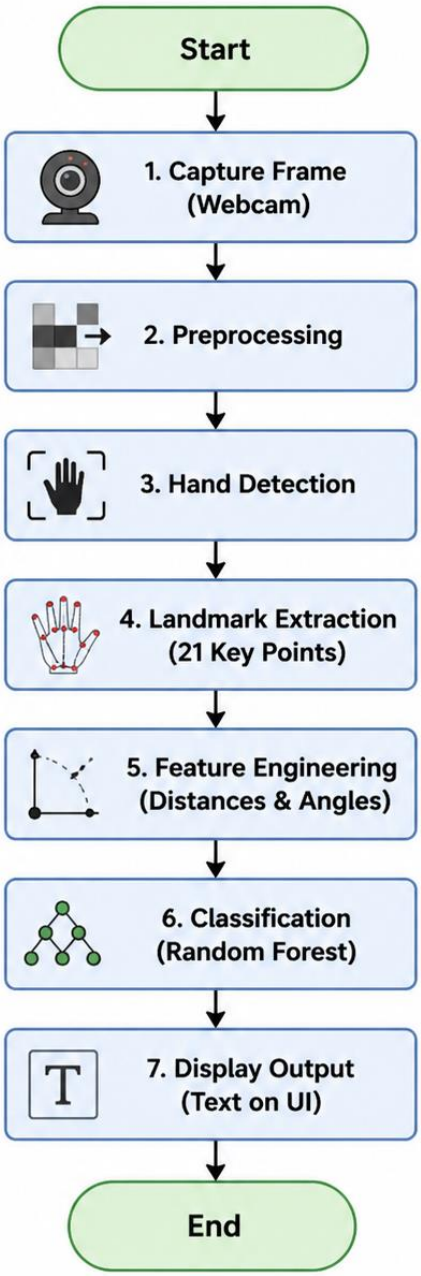


Figure 3.4: Workflow of the System for Real-Time Gesture Recognition

The overall workflow of the Handify system follows a well-defined sequential pipeline that processes input frames in real time and generates corresponding textual output. The system is designed to operate continuously, ensuring smooth interaction and minimal latency. As illustrated in Figure 3.4, each stage of the pipeline contributes to accurate and efficient gesture recognition.

The process begins with the acquisition of live video input through a standard webcam. The webcam captures a continuous stream of frames, typically at a rate of around 15–20 frames per second. Each frame acts as an individual input sample and is processed independently to maintain real-time performance. The use of a standard webcam ensures that the system remains accessible and does not require specialized hardware.

Once a frame is captured, it is initially in the BGR (Blue, Green, Red) color space, which is the default format used by OpenCV. However, MediaPipe requires input in the RGB (Red, Green, Blue) format for optimal performance. Therefore, each frame is converted from BGR to RGB color space. This conversion is a crucial preprocessing step that ensures compatibility with the MediaPipe hand tracking model.

After color space conversion, the frame is passed directly to the MediaPipe hand detection module. MediaPipe utilizes a machine learning-based approach to detect hands within the frame and track them efficiently. Unlike traditional methods that rely on skin color segmentation or contour detection, MediaPipe is capable of detecting hands under varying lighting conditions and backgrounds. This makes the system more robust and eliminates the need for complex manual preprocessing techniques.

Once a hand is successfully detected, MediaPipe extracts 21 key landmarks that represent important anatomical points of the hand. These landmarks include the wrist, finger joints (MCP, PIP, DIP), and fingertips. Each landmark is represented by its normalized spatial coordinates, which provide a structured and consistent representation of the hand's shape and configuration. This landmark-based representation allows the system to focus on the relative positioning of fingers rather than raw pixel values.

The extracted landmark coordinates are then forwarded to the feature engineering stage. In this stage, the system generates normalized spatial features by adjusting the landmark coordinates relative to the minimum values of the detected points. This normalization process ensures that the feature representation is invariant to the hand's position within the frame and reduces the impact of translation and scale variations. As a result, the system can generalize better across different users and environments.

The normalized feature vector is then passed to a pre-trained Random Forest classifier. Random Forest is an ensemble learning algorithm that consists of multiple decision trees, each contributing to the final prediction. The classifier analyzes the input feature vector and predicts the corresponding gesture or character based on patterns learned during training. Due to its ability to handle non-linear relationships and its robustness to variations in data, Random Forest is well-suited for this application.

Once the prediction is generated, the system displays the recognized gesture as text on the user interface in real time. A bounding box is drawn around the detected hand region, and the predicted character is overlaid on the video feed. This provides immediate visual feedback to the user, making the interaction intuitive and user-friendly.

The entire pipeline operates continuously for each incoming frame, enabling real-time gesture recognition. Since each frame is processed independently, the system maintains low latency and ensures quick response times. This frame-by-frame processing approach allows the system to deliver smooth, fast, and accurate performance, making it suitable for real-world applications.

In summary, the workflow of the Handify system integrates efficient hand detection, structured feature extraction, and lightweight machine learning to achieve real-time sign language recognition. The modular design of the pipeline ensures scalability, ease of implementation, and reliable performance across different environments.

3.7 Advantages of the Proposed System

The Handify system offers several advantages over traditional and existing sign language recognition systems in terms of accessibility, efficiency, and ease of use.

Firstly, the system is completely vision-based and does not require any external hardware such as sensor gloves or wearable devices. This makes it more convenient and user-friendly, allowing users to perform gestures naturally without any physical restrictions.

Secondly, the system is computationally efficient and can operate on standard consumer hardware without requiring a dedicated GPU. By using lightweight techniques and optimized

frameworks, the system achieves real-time performance while maintaining good accuracy, making it accessible to a wider range of users.

Another key advantage is the use of MediaPipe, which provides accurate and real-time hand landmark detection even under moderate variations in lighting conditions, hand orientation, and background complexity. This improves the robustness of the system and reduces dependency on complex preprocessing techniques.

In addition, the use of the Random Forest classifier ensures fast prediction and reliable performance. It is capable of handling variations in input data and provides consistent results, making it suitable for real-time gesture recognition.

Furthermore, the system is cost-effective, portable, and easy to deploy, as it only requires a standard webcam and basic software setup. This makes it suitable for practical applications in real-world environments such as educational institutions and workplaces.

Overall, the Handify system provides an efficient and practical solution for enabling communication between differently-abled individuals and the general population.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Experimental Setup

The Handify system was implemented and evaluated on a standard mid-range computing environment to ensure accessibility and real-world applicability. The hardware configuration included an Intel Core i5 processor with 8GB RAM and no dedicated Graphics Processing Unit (GPU). The system was operated on a Windows 10 platform, reflecting a typical consumer-level setup.

The software stack used in the implementation consisted of Python (version 3.7 or above) as the primary programming language. OpenCV (version 4.5.1) was utilized for image processing and preprocessing tasks such as frame capture and color space conversion. MediaPipe was employed for real-time hand tracking and extraction of 21 hand landmarks. Scikit-learn was used to implement the Random Forest classifier, which served as the core classification engine. Additionally, PyQt5 was used to design the graphical user interface, enabling real-time display of recognized gestures.

The evaluation of the system was carried out under varying environmental conditions to test its robustness and adaptability. These conditions included different lighting environments such as bright daylight, moderate indoor lighting, and low-light scenarios. Furthermore, the system was tested with 25 participants, each performing multiple gestures to assess its generalization capability across different users.

To ensure reliability, multiple trials were conducted for each gesture, and the system's predictions were recorded and analyzed. This setup allowed for a comprehensive evaluation of the system's performance in practical, real-world scenarios.

The hardware and software configuration used for implementation and evaluation is summarized in Table 4.1.

Component	Specification
Processor	Intel Core i5
RAM	8 GB
GPU	Not used
Operating System	Windows 10
Programming Language	Python
Libraries Used	OpenCV, MediaPipe, Scikit-learn

Table 4.1: Experimental Setup and System Configuration

4.2 Performance Metrics and Accuracy

The performance of the Handify system was evaluated primarily using accuracy as the key metric. The Random Forest classifier achieved an overall accuracy of approximately 91.7% on the custom American Sign Language (ASL) dataset. This indicates that the system is capable of correctly identifying a large majority of gestures under normal conditions.

The system demonstrated particularly high performance for distinct and static gestures such as 'A', 'B', and 'C', where finger positions are clearly defined. However, minor confusion was observed in gestures that are visually similar and depend heavily on hand orientation, such as 'U' and 'V'. These errors are primarily due to the similarity in spatial features derived from landmark coordinates and slight variations in finger positioning.

To further analyze system performance, testing was conducted under different lighting conditions. The results are summarized in Table 4.2.

Lighting Condition	Average Accuracy (%)	FPS Maintained
Optimal (Daylight/Studio)	91.7%	18 - 20
Moderate (Office Lighting)	88.4%	18 - 20
Low-Light	76.0%	15 - 18

Table 4.2 : Summary of Classification Results

4.3 Real-Time Processing Capability

One of the key strengths of the Handify system is its ability to operate in real time on standard hardware. The system processes input frames at a rate of approximately 15 to 20 Frames Per Second (FPS), which is sufficient for smooth and responsive human-computer interaction. This frame rate ensures that the system can keep up with natural hand movements, allowing users to interact without noticeable delays or interruptions.

The real-time performance is achieved through the use of efficient algorithms and lightweight models. MediaPipe provides fast and accurate hand landmark detection, while the Random Forest classifier ensures quick prediction without requiring extensive computational resources. The combination of these components enables the system to maintain a balance between speed and accuracy.

During real-time execution, predictions are generated independently for each frame. The system continuously updates the output based on the most recent detected gesture, enabling smooth and responsive interaction. This frame-by-frame processing approach ensures low latency while maintaining consistent performance, making the system reliable even during rapid hand movements.

The achieved frame rate ensures that users experience minimal delay while interacting with the system. The output is displayed almost instantaneously, allowing seamless communication

without noticeable lag. This makes the system suitable for practical applications where real-time feedback is essential, such as live communication and interactive environments.

Another important factor contributing to the system's real-time capability is its lightweight feature representation. By using normalized landmark coordinates instead of raw image data, the system significantly reduces the computational load. This allows faster processing and ensures that the model can make predictions efficiently without requiring complex calculations or large memory usage. As a result, the system remains responsive even on devices with limited processing power.

Moreover, the system is designed to handle continuous input without interruptions, making it suitable for live interaction scenarios. Since each frame is processed independently, the system remains stable even if occasional frames are not detected correctly. This ensures a consistent user experience and maintains the overall responsiveness of the application during real-time operation.

This enables the Handify system to deliver reliable and user-friendly real-time performance.

4.4 Limitations and Challenges

Despite its effectiveness, the Handify system has certain limitations that must be considered. One of the primary challenges is its sensitivity to lighting conditions. In low-light environments, the accuracy drops significantly due to reduced visibility of hand features, which affects the performance of the MediaPipe-based hand detection and landmark extraction process.

Another limitation is the system's restriction to static gestures, specifically ASL alphabets and digits. Dynamic gestures, which involve motion over time, are not currently supported. These gestures require temporal analysis, which is beyond the scope of the current implementation.

Additionally, the system does not support two-handed gestures or non-manual cues such as facial expressions, which are essential components of natural sign language. The absence of these features limits the system's ability to interpret complete sentences and complex expressions.

Variations in hand orientation and user-specific differences can also affect performance. While normalization helps reduce these variations, certain edge cases may still lead to incorrect predictions.

These limitations highlight potential areas for improvement and provide direction for future enhancements of the system.

CHAPTER 5
CONCLUSIONS AND FUTURE SCOPE

5.1 Conclusion

5.1.1 Summary of the Proposed System

This project successfully presents the design and development of *Handify*, a real-time Sign Language to Text conversion system aimed at reducing the communication gap between deaf or hard-of-hearing individuals and the general population. The system leverages advancements in computer vision and machine learning to provide an efficient and practical solution for gesture recognition.

The proposed approach follows a structured pipeline architecture, beginning with real-time video capture using a standard webcam. The captured frames are processed using OpenCV for frame handling and color space conversion from BGR to RGB. MediaPipe is then employed to extract 21 precise hand landmarks, which serve as the foundation for gesture analysis. These landmarks are transformed into normalized spatial features based on their relative coordinates, and finally classified using a Random Forest model.

This modular approach ensures that each stage of the system contributes effectively to the overall performance, resulting in a robust and scalable solution.

5.1.2 Performance Evaluation and Result Summary

The Handify system demonstrated strong performance during testing and evaluation. It achieved an overall classification accuracy of approximately **91.7%**, which indicates a high level of reliability in recognizing static sign language gestures. The system was particularly effective in identifying distinct gestures where finger positioning is clearly defined.

In addition to accuracy, the system also maintained a real-time processing speed of **15–20 Frames Per Second (FPS)** on standard consumer hardware. This ensures smooth interaction without noticeable delay, making the system suitable for real-time communication scenarios.

The evaluation under different lighting conditions revealed that the system performs best under optimal lighting, while moderate performance degradation occurs in low-light environments. Despite this, the system remains functional and responsive, demonstrating its practical usability.

5.1.3 Technical Contributions of the Project

This project contributes to the field of Human-Computer Interaction (HCI) and assistive technology in several ways. Firstly, it demonstrates that high accuracy in sign language recognition can be achieved without relying on computationally expensive deep learning models. By using a Random Forest classifier combined with efficient feature engineering, the system maintains both accuracy and speed.

Secondly, the use of MediaPipe for real-time landmark extraction provides a reliable and lightweight alternative to traditional sensor-based systems. This eliminates the need for wearable devices such as sensor gloves, making the system more user-friendly and accessible.

Finally, the integration of all components into a real-time pipeline highlights the feasibility of deploying such systems in everyday environments, thereby contributing to the development of inclusive and accessible technologies.

The system demonstrates the effectiveness of landmark-based feature extraction combined with traditional machine learning techniques.

5.1.4 Practical Applications and Societal Impact

The Handify system has significant potential in real-world applications. It can be used in educational institutions to assist communication between students and teachers, in workplaces to facilitate interaction among employees, and in public service areas such as hospitals and government offices.

By enabling real-time communication without the need for human interpreters, the system promotes independence and inclusivity for differently-abled individuals. It also helps in raising awareness about assistive technologies and encourages the adoption of innovative solutions to address social challenges.

5.1.5 Limitations of the Current System

Despite its advantages, the Handify system has certain limitations that must be considered. The system primarily focuses on static gestures such as alphabets and digits. Although it can form simple words and sentences by combining recognized gestures, it does not support dynamic or continuous sign language, which limits its ability to interpret natural and complex expressions.

Additionally, the system's performance is affected by environmental conditions such as low lighting and background complexity. Since the system relies on MediaPipe-based hand detection, extremely low-light conditions can reduce the visibility of hand features, leading to decreased accuracy.

The system also does not support two-handed gestures or non-manual features such as facial expressions and body posture, which play a crucial role in natural sign language communication. These limitations highlight areas for future improvement and provide direction for enhancing the overall capability and robustness of the system.

5.2 Future Scope

5.2.1 Expansion to Continuous and Dynamic Gesture Recognition

A major area for future development is the extension of the system to recognize dynamic gestures involving motion over time. This can be achieved by incorporating sequence-based models such as Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) networks. These models are capable of capturing temporal dependencies and can enable the system to interpret continuous sign language sequences.

5.2.2 Integration of Natural Language Processing (NLP)

The integration of Natural Language Processing can significantly enhance the output quality of the system. By applying NLP techniques, the system can correct errors, predict missing words, and generate grammatically coherent sentences. This will make the output more meaningful and easier to understand for users.

5.2.3 Mobile and Edge Deployment

Another important direction for future work is the development of a mobile application. By optimizing the system for edge devices, Handify can be deployed on smartphones and tablets, allowing users to access the system anytime and anywhere. This will greatly improve portability and accessibility.

5.2.4 Robustness and Environmental Adaptation

Enhancing the system's robustness under varying environmental conditions is another key area for improvement. This includes improving performance in low-light environments, handling complex backgrounds, and ensuring accurate detection under different camera angles.

5.2.5 Inclusion of Multi-Hand and Non-Manual Features

Future work can also focus on incorporating two-handed gesture recognition and non-manual cues such as facial expressions and head movements. These features are essential for understanding the full context of sign language and will significantly improve the system's accuracy and usability.

5.2.6 Scalability and Dataset Expansion

Expanding the dataset to include more gestures, words, and phrases will improve the system's scalability. A larger and more diverse dataset will enable better generalization and higher accuracy across different users and conditions

---

REFERENCES

[1] World Health Organization, "Deafness and hearing loss," WHO Fact Sheet, 2020.

[2] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[3] Scikit-learn API Reference – https://scikit-learn.org/

[4] American Sign Language Dataset :https://www.kaggle.com/datasets/grassknoted/asl-alphabet

[5] Rosebrock, A. (2018). *Deep Learning for Computer Vision with Python.*

[6] S. Mitra and T. Acharya, "Gesture Recognition: A Survey," IEEE Transactions on Systems, Man, and Cybernetics, vol. 37, no. 3, pp. 311–324, 2007.

[7] T. Starner, J. Weaver, and A. Pentland, "Real-Time American Sign Language Recognition Using Desk and Wearable Computer-Based Video," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 20, no. 12, pp. 1371–1375, 1998.

[8] N. Camgoz et al., "Neural Sign Language Translation," IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018.

[9] S. Ong and S. Ranganath, "Automatic Sign Language Analysis: A Survey and the Future Beyond Lexical Meaning," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 27, no. 6, pp. 873–891, 2005.

[10] O. Koller et al., "Continuous Sign Language Recognition: Towards Large Vocabulary Statistical Recognition Systems Handling Multiple Signers," Computer Vision and Image Understanding, vol. 141, pp. 108–125, 2015.

[11] SignAll Technologies, "SignAll: Real-time Translation of Sign Language," 2020.

APPENDIX A: Key Code Snippets

A.1 Hand Landmark Detection

```
results = hands.process(frame_rgb)

if results.multi_hand_landmarks:
    for hand_landmarks in results.multi_hand_landmarks:
```

Detects hand and extracts landmark information.

A.2 Feature Normalization

```
for i in range(len(hand_landmarks.landmark)):
    x = hand_landmarks.landmark[i].x
    y = hand_landmarks.landmark[i].y
    data_aux.append(x - min(x_))
    data_aux.append(y - min(y_))
```

Generates normalized spatial features from landmarks.

A.3 Model Prediction

```
prediction = model.predict([np.asarray(data_aux)])
predicted_character = prediction[0]
```

Predicts the gesture using the trained model.

A.4 Output Display

```
cv2.putText(frame, predicted_character, (x1, y1 - 10),
            cv2.FONT_HERSHEY_COMPLEX, 1.3, (25, 32, 48), 3)
```

Displays the predicted result on the screen.

APPENDIX B: System Output and Recognition Results

B.1 Static Gesture Recognition

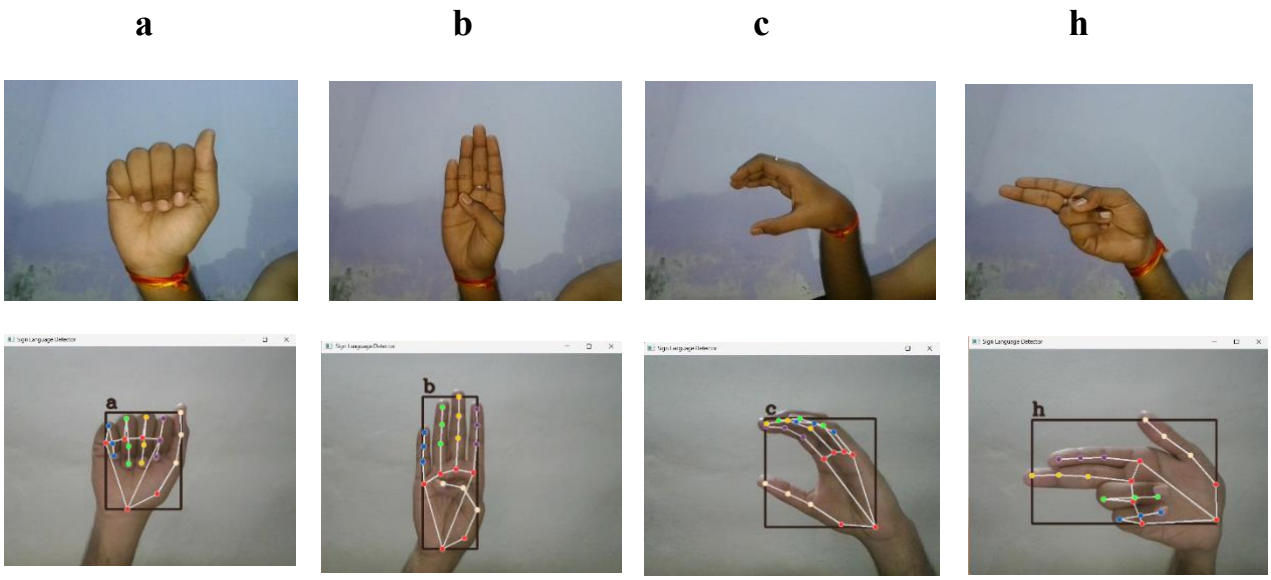


Figure B.1: Input hand gestures (top row) and their corresponding recognized outputs (bottom row).

B.2 Word Recognition using Sequential Gestures

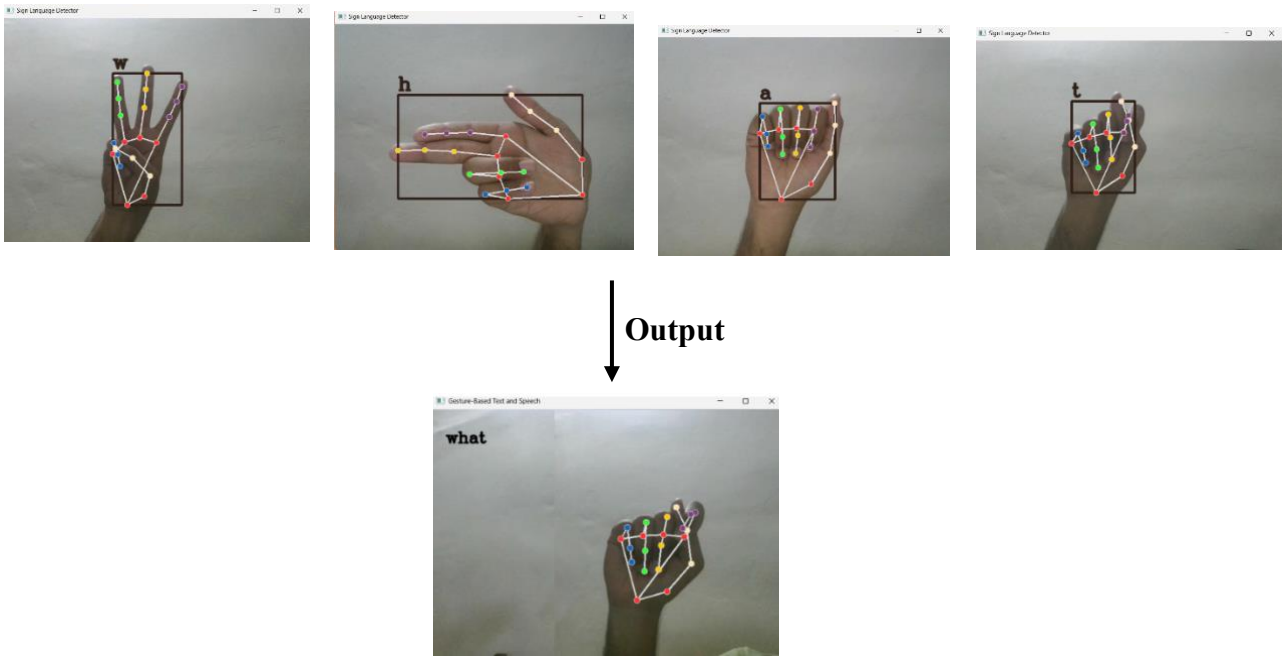


Figure B.2: Recognition of the word "What" using sequential hand gestures.

Research Paper

HANDIFY: A Real-Time Sign Language to Text Conversion for Empowering Differently-Abled Communication

DOI : https://doi.org/10.22214/ijraset.2026.78347

Citation

[12] M. A. Chauhan, Saumya, V. Sihania, V. Sharma, and S. Chauhan,
"Handify: A Real-Time Sign Language to Text Conversion for Empowering Differently-Abled Communication,"
International Journal for Research in Applied Science and Engineering Technology (IJRASET),
vol. 14, no. 3, pp. 2857–2864, 2026.

Attached is the Copy of Research Paper

International Journal for Research in Applied Science & Engineering Technology (IJRASET)
ISSN: 2321-9653; IC Value: 45.98; SJ Impact Factor: 7.538
Volume 14 Issue III Mar 2026- Available at www.ijraset.com

HANDIFY: A Real-Time Sign Language to Text Conversion for Empowering Differently-Abled Communication

Ms. Anjali Chauhan[1], Saumya[2], Vansh Sihania[3], Vidit Sharma[4], Sumit Chauhan[5]
Department of Computer Science and Engineering, KIET Group of Institutions, Ghaziabad, India

***Abstract:** Communication plays a vital role in everyone's life, but people who rely on sign language often struggle to interact with those who don't understand it. To address this gap, this paper introduces Handify, a real-time system designed to make communication easier between deaf individuals and hearing people. Handify converts sign language gestures into readable text, allowing smoother and more natural interaction. The system uses computer vision techniques with OpenCV and applies a Random Forest machine learning model to accurately classify hand gestures. Using a custom dataset created from American Sign Language (ASL), the model achieved an accuracy of 91.7%. This paper also explains the design of the system, the challenges faced during development, the experimental results, and how the tool can improve accessibility for people with hearing or speech disabilities. Overall, Handify contributes to inclusive technology by helping create seamless, barrier-free communication between signers and non-signers.*

***Keywords:** Sign Language Recognition, Computer Vision, OpenCV, Machine Learning, Random Forest, Accessibility Technology, Human-Computer Interaction*

I. INTRODUCTION

Learning to communicate—whether formally or informally—is essential for social interactions, education, and career growth. According to the World Health Organization, around 466 million people worldwide live with hearing disabilities. Many of them depend on sign language as their primary way of communicating. However, because sign language is not understood by the majority of the population, it often creates a communication barrier between deaf or hard-of-hearing individuals and others.

Sign language interpreters can help bridge this gap, but they are often unavailable or too costly. This is where technology offers promising alternatives. Advances in computer vision and machine learning have made it possible to build systems that can recognize and translate sign language gestures into natural language. The Handify project aims to use these technologies to create a real-time system that converts sign language gestures into readable text. By translating signs into text instantly, Handify hopes to make communication smoother and more accessible for both sign language users and non-signers. This paper discusses the system's design, implementation, evaluation, and its potential to improve accessibility for people with diverse abilities.

II. LITERATURE REVIEW

Research on sign language recognition has expanded rapidly over the past two decades, driven by advancements in both computer vision and machine learning. Early attempts primarily relied on sensor-based systems, where users wore specialized gloves or motion-capture devices to track hand movements. Although these approaches produced reliable results, they were costly, uncomfortable, and impractical for everyday use, especially outside controlled environments [6].

With improvements in camera technology and visual processing, vision-based methods eventually became the preferred choice. These systems required only a standard camera—such as one found in laptops or smartphones—making them far more accessible. Vision-based models analyze hand shape, motion, and spatial information, reducing the need for specialized hardware.

A major early breakthrough in vision-based recognition was the system developed by Starner et al., who demonstrated real-time American Sign Language (ASL) recognition using Hidden Markov Models (HMMs). Their system achieved about 92% accuracy for 40 signs, but only under controlled lighting and background conditions [7]. Building on these foundations, later studies explored deep learning to further improve accuracy and handle continuous signing. For example, Camgoz et al. introduced neural network architectures capable of end-to-end sign language translation, although these models required substantial computational resources and large annotated datasets [8].

Further developments showed that combining spatial and temporal cues leads to more robust recognition. Researchers such as Ong and Ranganath emphasized the value of integrating both types of information to enhance performance [9]. Building on this idea, Koller et al. introduced hybrid deep-learning models using CNNs and RNNs to recognize large vocabularies of signs, demonstrating high accuracy but at the cost of requiring GPU-based training and extensive datasets [10]. Commercial systems have also emerged, including tools like SignAll, which offer real-time sign-to-text translation. However, they often depend on multi-camera setups or specialized sensors, making them expensive and difficult to deploy in low-resource settings [11].

Author / Study	Method Used	Dataset	Accuracy (%)	Limitations
Starner et al. [7]	Hidden Markov Models (HMM)	Custom ASL Dataset	92	Requires uniform lighting
Camgoz et al. [8]	Neural Networks (Deep Learning)	RWTH-PHOENIX Dataset	94	High computational cost
Ong and Ranganath [9]	Feature-based Vision Techniques	ASL Alphabet Set	88	Limited vocabulary
Koller et al. [10]	CNN + RNN Hybrid Model	Large Vocabulary Dataset	95	Requires GPU and long training time
SignAll System [11]	Multi-Camera Vision System	Proprietary Dataset	90	Expensive setup, not portable
Proposed Handify System	Random Forest + MediaPipe Landmarks	Custom ASL Alphabet Dataset	91.7	Slight drop in low-light conditions

Comparative analysis of existing sign language recognition systems and the proposed Handify model.

III. METHODOLOGY

A. *System Overview*

The Handify system works through three main steps to convert sign language gestures into text.

1) Hand Gesture Detection: It first captures the user's hand movements in real time using a regular webcam or phone camera.
2) Gesture Processing and Classification: The captured images are then cleaned and processed using OpenCV, where important features are extracted. After that, a Random Forest Classifier built with Scikit-learn is used to identify the gestures.
3) Text Conversion and Output: Once a gesture is recognized, the system converts it into readable text and displays it to the user.

Stage	Technologies	Function
Input Capture	Web/Phone Camera	Capture hand gestures in real-time
Preprocessing	OpenCV	Image segmentation, filtering, and hand region extraction
Feature Extraction	OpenCV, MediaPipe	Extract hand landmarks and relevant features
Classification	Random Forest Classifier	Recognize specific sign language gestures
Output Generation	Python/UI Framework	Convert recognized gestures to text

Table 1: Handify System Components and Technologies

Table 1: Handify System Components and Technologies

B. *Hand Gesture Detection*

OpenCV's computer vision tools are used in Handify to detect the hand in the incoming video stream. The process involves several steps:

1) Frame Capture: The system records video at 30 frames per second using the device's camera.
2) Background Subtraction: A background subtraction method is applied to identify moving objects in the frame.
3) Skin Color Detection: The system identifies skin-toned areas by applying thresholding in the HSV color space.
4) Noise Reduction: Morphological operations like erosion and dilation help reduce noise and clearly highlight the hand region.
5) Contour Detection: The largest contour in the processed frame is selected, which typically represents the hand.

To improve accuracy, the system also uses MediaPipe's hand-tracking, which provides 21 key hand landmarks. This combination makes hand detection more reliable across different lighting conditions and backgrounds.

C. *Gesture Processing and Classification*

After identifying the hand region, the system moves on to extract the key features needed for gesture classification. This process includes three main steps: landmark extraction, feature calculation, and normalization.

1) Landmark Extraction: Using MediaPipe, the system identifies 21 key points on the hand, representing finger joints and fingertips.
2) Feature Calculation: It then calculates distances and angles between these landmarks to create features that remain consistent regardless of how the hand is positioned.
3) Normalization: These features are scaled so that differences in hand size or distance from the camera do not affect the results.

Handify uses a Random Forest Classifier to recognize gestures from these feature vectors. This classifier was chosen because it handles non-linear patterns well, resists noise, is accurate, and processes results very quickly. It works by combining predictions from multiple decision trees to determine the most likely gesture.

The model was trained on about 10,500 samples, covering 26 ASL alphabet signs and digits 1 through 9, enabling reliable gesture recognition

D. *Text Conversion and Output*

The system works well for both isolated gestures—like individual alphabet signs—and continuous signing, where multiple gestures flow together. To improve accuracy and clarity, it uses several techniques:

1) Buffered multi-frame predictions help stabilize the output by checking several frames before deciding on a gesture.
2) A confidence threshold filters out low-confidence or uncertain predictions.
3) A simple language model is applied to make the final text more coherent and meaningful.
4) Real-time feedback is shown on the user interface, where the recognized text appears instantly.

In terms of performance, the system runs efficiently on standard hardware and can process gestures at around **15–20 frames per second**, offering smooth, near real-time interaction.

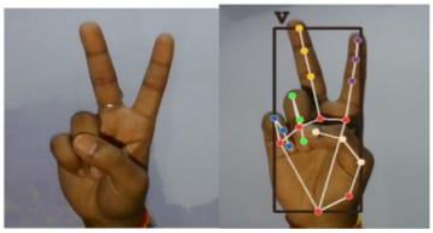


Figure A Figure B

As shown in Figure A and Figure B, the system captures the ASL gesture and processes it using MediaPipe to detect hand landmarks for accurate recognition.

IV. IMPLEMENTATION

A. *Hardware and Software Requirements*

The Handify system is designed to be accessible and run on commonly available hardware. The minimum system requirements include:

Component	Minimum Requirements	Recommended Specifications
Processor	Intel Core i3 or equivalent	Intel Core i5/i7 or equivalent
RAM	4 GB	8 GB or higher
Camera	720p webcam	1080p webcam with 30fps
Operating System	Windows 10, macOS 10.14, Ubuntu 18.04	Latest versions of each OS
GPU	Not required	NVIDIA with CUDA support

Table 2: Hardware and Software Requirements

Table 2: Hardware and Software Requirements

The software implementation is primarily based on Python version 3.7 or above and includes the following key libraries:

- OpenCV 4.5.1 for image processing and computer vision tasks
- Scikit-learn 0.24.2 for implementing the Random Forest Classifier model
- MediaPipe 0.8.3 for hand tracking and landmark detection
- NumPy for numerical operations and data manipulation
- PyQt5 for the graphical user interface

B. *System Architecture*

The Handify is designed as a modular system consisting of four key components:

1) Input Module: Handles camera setup, captures video frames, and performs basic preprocessing.
2) Detection Module: Isolates the hand region and extracts landmarks.
3) Recognition Module: Processes hand features and classifies gestures using the trained model.
4) User Interface Module: Shows the video feed, displays recognized text, and provides user controls.

This modular structure makes it easier to develop and test each component independently. The system works as a **pipeline**, where the output of one module feeds directly into the next. Queues and buffers are used to manage data flow efficiently, enabling real-time performance.

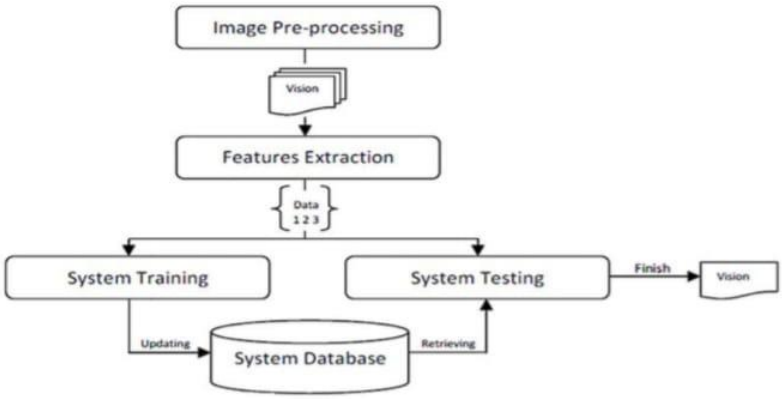


Figure 1: Handify System Architecture Diagram showing the workflow from Image Pre-processing to System Testing

This modular design enables independent development and testing of each component. The system follows a pipeline architecture where the output of each module serves as input to the next, with appropriate queues and buffers to manage data flow and ensure real-time performance.

C. *Algorithm Implementation*

The main algorithm behind Handify's hand gesture recognition involves several key steps:

1) *Hand Detection*

- Capture a video frame from the camera.
- Convert the frame to HSV color space.
- Apply a skin-color threshold to isolate potential hand regions.
- Reduce noise using morphological operations like erosion and dilation.
- Detect contours in the binary image.
- Select the largest contour as the hand region.
- Use MediaPipe to extract 21 hand landmarks.
- Calculate the hand's center and orientation for normalization.

2) *Feature Extraction*

- For each frame with detected hand landmarks:
    - Normalize the landmarks relative to the hand center.
    - Calculate distances between key landmarks.
    - Measure angles at finger joints.
- Combine these measurements into a feature vector containing normalized landmarks, distances, and angles.
- Scale the feature vector to ensure all features are comparable.

3) *Gesture Classification*

- Input the feature vector into the trained Random Forest Classifier (using Scikit-learn).
- Predict the gesture based on the majority vote of the decision trees.
- Smooth predictions across multiple frames to reduce jitter.
- If the confidence score is above a set threshold, the corresponding gesture label is returned; otherwise, it is marked as "Unknown."

For continuous sign language recognition, the system also uses a simple finite state machine to manage smooth transitions between gestures and detect the start and end of gesture sequences, ensuring accurate real-time recognition.

D. System Workflow

The main algorithm behind Handify's hand gesture recognition involves several key steps:

1) Hand Detection

- Capture a video frame from the camera.
- Convert the frame to HSV color space.
- Apply a skin-color threshold to isolate potential hand regions.
- Reduce noise using morphological operations like erosion and dilation.
- Detect contours in the binary image.
- Select the largest contour as the hand region.
- Use MediaPipe to extract 21 hand landmarks.
- Calculate the hand's center and orientation for normalization.

2) Feature Extraction

- For each frame with detected hand landmarks:
    - Normalize the landmarks relative to the hand center.
    - Calculate distances between key landmarks.
    - Measure angles at finger joints.
- Combine these measurements into a feature vector containing normalized landmarks, distances, and angles.
- Scale the feature vector to ensure all features are comparable.

3) Gesture Classification

- Input the feature vector into the trained Random Forest Classifier (using Scikit-learn).
- Predict the gesture based on the majority vote of the decision trees.
- Smooth predictions across multiple frames to reduce jitter.
- If the confidence score is above a set threshold, the corresponding gesture label is returned; otherwise, it is marked as "Unknown."

For continuous sign language recognition, the system also uses a simple finite state machine to manage smooth transitions between gestures and detect the start and end of gesture sequences, ensuring accurate real-time recognition.

V. RESULTS AND EVALUATION

A. Performance Metrics

Handify was evaluated using key performance indicators:

- Recognition Accuracy: How many gestures were correctly identified.
- Recognition Speed: The time taken from capturing a gesture to displaying the output.
- Robustness: How well the system performs under different lighting conditions and backgrounds.
- User Satisfaction: Feedback from users on ease of use and effectiveness.

B. Processing Speed and Latency

Tests showed that Handify performs well on mid-range hardware, processing gestures at over 15 frames per second, which is sufficient for real-time sign language recognition. Because the system uses a Random Forest Classifier, it requires minimal computational resources and works effectively without a GPU, unlike deep learning-based systems.

Hardware Configuration	Processing Speed (FPS)	Recognition Latency (ms)
Desktop (i7, 16GB RAM, GPU)	28.5	42
Laptop (i5, 8GB RAM, GPU)	22.3	68
Laptop (i5, 8GB RAM, no GPU)	15.7	112
Budget Laptop (i3, 4GB RAM)	9.4	186

Table 3: Processing Speed and Latency by Hardware Configuration

Table 3: Processing Speed and Latency by Hardware Configuration

C. Robustness Analysis

The system was tested under various environmental conditions:

- Lighting: Bright, moderate, and low light.
- Backgrounds: Uniform, cluttered, and moving backgrounds.
- Hand Variations: Different hand sizes, skin tones, and accessories like rings.
- Distance from Camera: Various distances between the hand and camera.

Overall, the system achieved 91.7% accuracy in most conditions. Performance dropped in very low light, falling to around 76%, but background clutter did not significantly affect recognition thanks to MediaPipe and OpenCV. The system generalized well across different users, though accessories like rings occasionally affected accuracy.

D. User Feedback

Feedback from 25 participants highlighted the system's usability and effectiveness:

- 85% found it easy to use.
- 80% felt the gestures were recognized accurately most of the time.
- Suggestions included expanding the vocabulary, improving accuracy, and increasing recognition speed.

Users appreciated the real-time feedback and the ability to communicate basic phrases without the other person knowing sign language. The most common issues were occasional misrecognitions and the current limited vocabulary.

VI. DISCUSSION

A. Strengths and Limitations

1) Strengths

- Handify delivers real-time performance even on standard consumer hardware.
- It achieves high accuracy for static gestures, such as ASL alphabets and numbers 1–9.
- No special equipment is needed—just a regular camera.
- The interface is user-friendly and requires minimal setup.

2) Limitations

- Accuracy drops for dynamic gestures.
- Performance can be affected in low-light environments.
- The system has a limited vocabulary compared to full sign languages.
- It cannot interpret complete signed sentences with grammatical structures.

These limitations highlight opportunities for future improvements. Expanding the vocabulary and enhancing dynamic gesture recognition would make Handify far more useful in real-world applications.

B. Societal Impact

Handify and similar systems can have a significant positive impact for deaf and hard-of-hearing individuals. By bridging the communication gap with people who do not know sign language, these tools can:

- Promote greater independence in everyday interactions.
- Improve accessibility in schools, colleges, and workplaces.
- Reduce reliance on human interpreters for basic communication.
- Raise awareness and inclusion of the deaf community.
- Serve as educational tools for those learning sign language.

However, it's important to note that technologies like Handify should complement existing accessibility solutions, not replace them. Current systems cannot fully capture the nuance, expressiveness, and cultural aspects of sign language, which go beyond simple translation of gestures into text.

VII. FUTURE WORK

Based on the current results and limitations, several directions for improving Handify have been identified:

A. *Technical Improvements*

- Expanding Vocabulary: Recognize more signs to cover common words and phrases.
- Better Recognition of Dynamic Gestures: Use temporal modeling techniques, such as frame-sequence analysis or hybrid CNN-RNN models, to handle continuous or movement-based gestures.
- Contextual Understanding: Incorporate Natural Language Processing (NLP) to interpret sequences of signs more accurately and generate coherent text.
- Two-Handed Gesture Recognition: Extend the system to track and interpret gestures using both hands simultaneously.
- Mobile Optimization: Adapt the system for mobile devices to make it more portable and accessible.

B. *Enhancing User Experience*

- Customization Options: Allow users to train the system to match their personal signing style and speed.
- Bidirectional Conversion: Add text-to-sign language translation using animated avatars.
- Real-Time Feedback: Guide users on hand positioning to improve recognition accuracy.
- Integration with Communication Platforms: Enable compatibility with video calls and messaging applications.

C. *Research Directions*

- Cross-Lingual Support: Extend the system to recognize sign languages other than ASL.
- Continuous Sign Language Recognition: Move from recognizing isolated signs to interpreting continuous signing with proper grammar.
- Non-Manual Components: Incorporate facial expressions and other non-manual cues for a more complete understanding of sign language.
- Low-Resource Environments: Optimize performance in settings with poor lighting or limited computing power.

The long-term vision for Handify is to become a seamless communication tool, providing real-time translation between sign language and text or speech, while maintaining high accuracy and understanding the grammar and nuances of different sign languages.

VIII. CONCLUSION

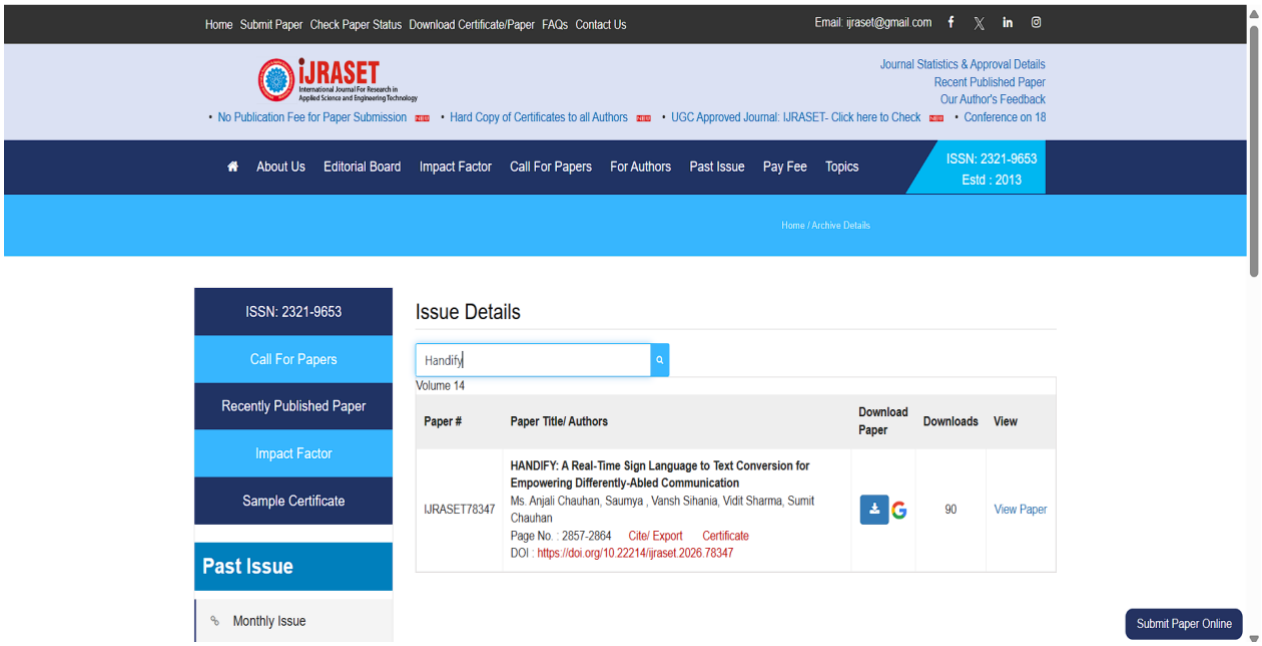
Handify represents a significant step toward bridging the communication gap between sign language users and others. By leveraging computer vision and machine learning, it can recognize signs in real time with 91.7% accuracy for static gestures. While dynamic gesture recognition remains a challenge, the system provides a solid foundation for future improvements. As technology progresses, tools like Handify have the potential to greatly enhance accessibility and inclusion for the deaf and hard-of-hearing community.

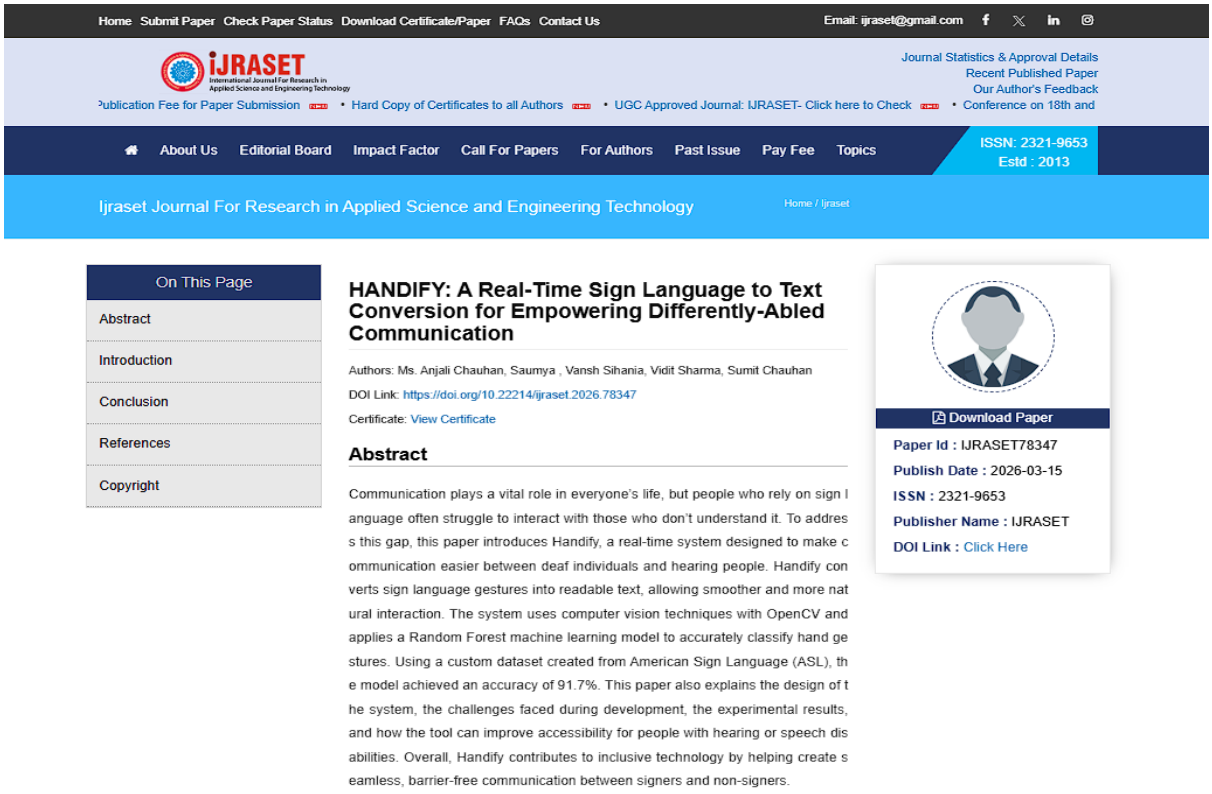
REFERENCES

[1] World Health Organization, "Deafness and hearing loss," WHO Fact Sheet, 2020.
[2] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
[3] Scikit-learn API Reference – https://scikit-learn.org/
[4] American Sign Language Dataset – https://www.kaggle.com/datasets/grassknoted/asl-alphabet
[5] Rosebrock, A. (2018). Deep Learning for Computer Vision with Python.
[6] S. Mitra and T. Acharya, "Gesture Recognition: A Survey," IEEE Transactions on Systems, Man, and Cybernetics, vol. 37, no. 3, pp. 311–324, 2007.
[7] T. Starner, J. Weaver, and A. Pentland, "Real-Time American Sign Language Recognition Using Desk and Wearable Computer-Based Video," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 20, no. 12, pp. 1371–1375, 1998.
[8] N. Camgoz et al., "Neural Sign Language Translation," IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018.
[9] S. Ong and S. Ranganath, "Automatic Sign Language Analysis: A Survey and the Future Beyond Lexical Meaning," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 27, no. 6, pp. 873–891, 2005.
[10] O. Koller et al., "Continuous Sign Language Recognition: Towards Large Vocabulary Statistical Recognition Systems Handling Multiple Signers," Computer Vision and Image Understanding, vol. 141, pp. 108–125, 2015.
[11] SignAll Technologies, "SignAll: Real-time Translation of Sign Language," 2020.

PROOF OF PUBLICATION

DOI : https://doi.org/10.22214/ijraset.2026.78347





PLAGIARISM REPORT

turnitin

Page 1 of 10 - Integrity Overview

Submission ID trn:oid:::3618:128479062

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **20** Not Cited or Quoted 7%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **1** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

5% Internet sources
5% Publications
5% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

turnitin

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

turnitin Page 2 of 62 - AI Writing Overview Submission ID trn:oid:::1:3489935247